Politecnico di Torino

eBPF: Verifier Vulnerabilities and Exploitation

Andrea Botticella s347291

Politecnico di Torino
Security Verification and Testing
eBPF Research Lab

June 29, 2026

Contents

1 Introduction

1.1 eBPF Background

Extended Berkeley Packet Filter (eBPF) allows user-space programs to safely execute custom logic inside the Linux kernel without needing custom modules. Because these programs run in ring-0, a single vulnerability could easily crash the system or lead to privilege escalation. To prevent this, every eBPF program must pass the **BPF verifier**, a static analysis engine inside the kernel that formally checks the program for memory safety and type correctness before JIT compilation.

This report documents an analysis of four significant logic bugs affecting the verifier, comparing their root causes, exploitation primitives, and the protective mechanisms that mitigate them.

1.2 Research Objectives

This project is part of the Security Verification and Testing course. The main goal is to analyze the patches, understand the vulnerabilities at a deep technical level, and develop proof-of-concept code for several verifier CVEs. While there are many vulnerabilities available for study, I have selected the following ones for this research:

- **CVE-2021-3600**: A 32-bit truncation bug during the verifier's div/mod instruction fixup, leading to bounds tracking confusion and Out-Of-Bounds (OOB) memory access.
- **CVE-2021-3444**: The sister bug of CVE-2021-3600, targeting the mod32 destination register truncation. Exploited on kernel 4.19.19 (where `alu_limit` is absent) to achieve a **full unprivileged local privilege escalation** via ret2usr.
- **CVE-2023-52452**: An integer tracking flaw regarding variable-offset stack writes, which incorrectly calculates the depth of stack frames and causes stack corruption.
- **CVE-2024-26589**: A missing bounds check inside the restricted pointer arithmetic mechanisms which allows arbitrary offset-addition to the `flow_keys` structures, leading to stack OOB.

1.3 Research Methodology

The exploitation process is organized into distinct, verifiable *Milestones*, starting from initial vulnerability confirmation (such as information leaks) up to more advanced exploitation stages and analyzing the technical feasibility of different attack vectors.

This modular structure ensures that the research methodology remains both robust and extensible for future vulnerability analyses, allowing for the seamless integration of additional CVE studies regardless of the specific subsystem or bug class involved.

1.4 Common Experimental Framework

To improve reproducibility and keep the methodology scalable as new CVEs are added, experiments are conducted using a shared framework and CVE-specific parameterization. This section defines what is common across analyses; concrete values (kernel release, config variants, and mitigation profile) are documented inside each CVE chapter.

Component	Common baseline
Build workflow	Buildroot-based pipeline orchestrated by project scripts
Virtualization	QEMU/KVM virtual testbed
Root filesystem strategy	Minimal Buildroot image with CVE overlays
Kernel strategy	Per-CVE vulnerable target kernel selected for validation
eBPF core requirements	`CONFIG_BPF_SYSCALL=y`, `CONFIG_BPF_JIT=y`
Debug and observability	Kernel symbols/debug options enabled when needed by the CVE
Mitigation profile handling	Controlled and documented per scenario (privileged/unprivileged)
Compiler/toolchain strategy	Static user-space binaries for deterministic VM execution
Execution model	Milestone-driven validation and incremental exploit development

Table 1.1: Shared experimental framework used across CVE analyses

This separation between common framework and per-CVE specifics keeps the report coherent, extensible, and directly reusable for future verifier vulnerability studies.

2 CVE-2021-3600

2.1 CVE Selection

I selected CVE-2021-3600 inspired by previous years' research projects, such as CVE-2020-8835, which pioneered scalar bounds confusion on BPF Maps. This case provides an architectural study of how post-verification instruction rewriting can undermine the safety proofs established by the verifier, and it allowed me to test how modern kernel mitigations, specifically the `alu_limit` pointer arithmetic sanitization, respond to a classic bounds tracking mismatch between the verifier and the JIT compiler.

2.2 Vulnerability Analysis

2.2.1 Root Cause

The root cause lies in `fixup_bpf_calls()` in `kernel/bpf/verifier.c`. This function is a post-verification pass that rewrites certain BPF instructions before JIT compilation. For 32-bit division and modulo operations, the vulnerable code inserts:

Vulnerable Code and Truncation Bug

```
struct bpf_insn mask_and_div[] = {
    BPF_MOV32_REG(insn->src_reg, insn->src_reg),  // BUG!
    /* Rx div 0 -> 0 */
    BPF_JMP_IMM(BPF_JNE, insn->src_reg, 0, 2),   // 64-bit jump!
    BPF_ALU32_REG(BPF_XOR, insn->dst_reg, insn->dst_reg),
    BPF_JMP_IMM(BPF_JA, 0, 0, 1),
    *insn,
};
```

Listing 2.1: Vulnerable fixup code (kernel/bpf/verifier.c, fixup_bpf_calls())

Why `BPF_MOV32_REG` is wrong: The `BPF_MOV32_REG(src, src)` instruction zeroes the upper 32 bits of the source register at *runtime*. However, the verifier's abstract interpretation of the program happens *before* `fixup_bpf_calls()` runs. The verifier never sees this inserted `mov32` instruction, so its bounds tracking for the source register remains unchanged, it still reflects the full 64-bit value.

Furthermore, during the actual `BPF_ALU | BPF_DIV | BPF_X` verification, the function `adjust_scalar_min_max_vals()` only marks the *destination* register as unknown after division, the source register's bounds are not updated.

The double bug: There is actually a second problem: the zero-check `BPF_JMP_IMM(BPF_JNE, src, 0, 2)` uses a **64-bit comparison**, even for 32-bit operations. After the `mov32` truncation, a value like `0xFFFFFFFF` (which is nonzero) would not trigger the zero-check. But the verifier, which still tracks the original 64-bit value, may compute the wrong result for the division.

Concrete Example:

```
// Source level:
0: (b7) r0 = -1           // R0 = 0xFFFFFFFF_FFFFFFFF
1: (b7) r1 = -1           // R1 = 0xFFFFFFFF_FFFFFFFF
2: (3c) w0 /= w1          // 32-bit division

// After fixup_bpf_calls() rewrites instruction 2:
2a: (bc) w1 = w1          // mov32: R1 -> 0x0000_0000_FFFF_FFFF
2b: (15) if r1 == 0 goto  // 64-bit zero check (R1 != 0, so not taken)
2c: (3c) w0 /= w1         // actual division
```

Listing 2.2: Bounds mismatch demonstration

Register	Verifier State	Runtime Value
R1 before div	`0xFFFFFFFF_FFFFFFFF` (-1)	`0x0000_0000_FFFF_FFFF` (after mov32)
R1 » 32	`0xFFFFFFFF`	`0x00000000`

Table 2.1: Verifier vs. runtime state after the truncation bug

2.2.2 Patch Details

The vulnerability is addressed by correcting the instruction rewriting logic in `fixup_bpf_calls()`. Specifically, the patch:

1. **Removes** the `BPF_MOV32_REG(src, src)` instruction insertion, eliminating the truncation of the source register at runtime.
2. **Replaces** the 64-bit `JMP_IMM` zero-check with a proper `JMP32` instruction for 32-bit operations.

Without the inserted `mov32`, the verifier's tracked bounds always remain synchronized with the actual runtime values.

Kernel series	Fixed in
5.4.x	5.4.101
5.10.x	5.10.19
5.11.x	5.11.2

Table 2.2: Backport versions for CVE-2021-3600

2.2.3 Exploiting the Bounds Mismatch

The key insight is creating a register whose verifier-tracked value differs from its runtime value after a 32-bit division:

- Set source register `R3 = 0xFFFFFFFF_00000001`
- Set target register `R2 = target_offset` (any value)
- Execute `w2 /= w3` (32-bit div)
- **Verifier**: `R2 = target / 0xFFFFFFFF_00000001` $\approx$ `0` (very large divisor $\rightarrow$ quotient near zero)
- **Runtime**: `R3` truncated to `0x00000001`, so `R2 = target / 1 = target` (unchanged!)

Now the verifier believes `R2` $\approx$ `0` (within map bounds), but at runtime `R2 = target_offset` which can be out of bounds. Using `R2` as an index into a map value pointer gives us an **out-of-bounds read or write**.

2.3 Environment Setup

Component	Details
Host OS	Linux (Ubuntu-based)
Virtualization	QEMU/KVM (Buildroot-based rootfs)
Kernel Version	5.10.15 (vulnerable)
Rootfs	Buildroot minimal with custom exploit overlay
Kernel Config	`CONFIG_BPF_SYSCALL=y`, `CONFIG_BPF_JIT=y` `CONFIG_KALLSYMS_ALL=y` KASLR disabled (`nokaslr`), SMEP/SMAP disabled
VM Resources	2 vCPUs, 512 MB RAM
Compilation	GCC, static linking, `-O2`
User	root (for Milestones 1–3), user uid=1000 (for M4 LPE)

Table 2.3: Test environment for CVE-2021-3600

Kernel version 5.10.15 was selected because the fix for CVE-2021-3600 was backported to the 5.10 stable branch starting from version 5.10.16. The vulnerable `mask_and_div[]` code with the buggy `BPF_MOV32_REG(src, src)` is present in 5.10.15. A matching kernel configuration file (`v5.10.15.config`) was created to ensure BPF subsystem options were correctly enabled.

2.4 Proof of Concept (PoC)

2.4.1 Milestone 1: Demonstrate Bounds Confusion

Objective Prove that the BPF verifier's bounds tracking becomes stale after a 32-bit division due to the inserted `BPF_MOV32_REG(src, src)`, demonstrating that the runtime value of the source register differs from what the verifier tracks.

Approach The BPF program:

1. Sets `R0 = R1 = -1` (`0xFFFFFFFF_FFFFFFFF`)
2. Saves a copy of `R1` to `R8` (before the fixup affects R1)
3. Executes `w0 /= w1` (32-bit division)
4. After division, right-shifts `R1 » 32` to expose the truncation
5. Exfiltrates the shifted value and the original `R8 » 32` to a BPF map

The key comparison:

- `R1 » 32` should be `0` at runtime (upper bits zeroed by mov32) but the verifier thinks it's `0xFFFFFFFF`
- `R8 » 32` should be `0xFFFFFFFF` (R8 was copied before the fixup)

Results

```
[+] BPF program LOADED (fd=5) - verifier accepted!

  result[0] (div result w0/w1)   = 0x0000000000000001  <- 0xFFFFFFFF / 0xFFFFFFFF = 1
  result[1] (R1 >> 32 after div) = 0x0000000000000000  <- upper bits ZEROED (BUG!)
  result[2] (orig R1 >> 32)      = 0x00000000ffffffff  <- original had upper bits

[+] VULNERABILITY CONFIRMED:
```

```
    The verifier's fixup_bpf_calls() inserted mov32 that
    truncated R1's upper 32 bits at runtime, but the verifier
    still tracks the original 64-bit bounds.
```

Listing 2.3: Milestone 1 output on vulnerable kernel 5.10.15

Analysis: Result[1] being 0 while result[2] is `0xFFFFFFFF` proves the `BPF_MOV32_REG(R1, R1)` zeroed R1's upper bits at runtime while the verifier continued tracking the original value. The verifier's bounds are stale. This milestone confirms the vulnerability is present and exploitable on kernel 5.10.15.

2.4.2 Milestone 2: OOB Read (KASLR Bypass)

Objective Leverage the bounds confusion to perform out-of-bounds reads past a BPF map value boundary, leaking kernel heap data.

Approach

1. Create a BPF array map with `value_size = 256` bytes
2. Lookup the map value to get `PTR_TO_MAP_VALUE` in `R6`
3. Set `R2 = oob_offset` (target offset beyond the 256-byte boundary)
4. Set `R3 = 0xFFFFFFFF_00000001` via 64-bit immediate load
5. Execute `w2 /= w3`:
    - Verifier: `R2 = oob_offset / 0xFFFFFFFF_00000001 = 0` (within bounds)
    - Runtime: `R3` truncated to 1, so `R2 = oob_offset` (OOB!)
6. Compute `R7 = R6 + R2` (map pointer + confused index)
7. Read `*(R7 + 0)`, verifier allows (thinks offset = 0), runtime reads OOB
8. Exfiltrate leaked data to result map

For each of 15 OOB offsets (256, 264, 272, ..., 368), a separate program is loaded and run.

Results

```
[*] === CVE-2021-3600 Milestone 2: OOB Read via Bounds Confusion ===
[+] Data map created (fd=3, value_size=256)
[+] Result map created (fd=4)
[*] Attempting OOB reads beyond map value (boundary at +0x100)...

  leaked[ 0] (+0x100) = 0x0000000000000000
  leaked[ 1] (+0x108) = 0x0000000000000000
  ...
  leaked[14] (+0x170) = 0x0000000000000000

  Total OOB reads: 15/15 successful
```

Listing 2.4: Milestone 2 output on kernel 5.10.15 (as root)

Analysis: The OOB reads return `0x0000000000000000`, zero-filled slab padding beyond the 256-byte map value. This is **different** from the map fill value (`0x4141414141414141`), confirming that the BPF program genuinely accesses memory past the map value boundary. The leaked data corresponds to the SLUB allocator's zero-initialized padding after the map value allocation, which in a more densely populated heap would contain kernel pointers and metadata from adjacent allocations.

2.4.3 Milestone 3: OOB Write + Escalation Analysis

Objective Extend the bounds confusion to write operations and analyze the feasibility of privilege escalation.

Approach The same bounds confusion used for reads also works for writes:

1. Obtain the confused index `R2` (verifier: 0, runtime: OOB offset)
2. Compute `R7 = map_ptr + R2`
3. Write a marker value: `*(R7) = 0xDEAD1337CAFE4242`
4. Read back the written value to confirm the OOB write
5. Run a separate program to independently verify persistence
6. Repeat across 5 rounds for reliability testing

Results

```
Part 1: OOB Write
  Write program read-back: 0xdead1337cafe4242  <- marker confirmed
  Independent read:        0xdead1337cafe4242  <- OOB WRITE CONFIRMED

Part 2: Reliability test (5 rounds)
  Run 1/5: PASS
  Run 2/5: PASS
  ...
  Reliability: 5/5 successful (100%)
```

Listing 2.5: Milestone 3 output on kernel 5.10.15

Escalation Analysis
The combined OOB read (Milestone 2) and OOB write (Milestone 3) primitives provide:

Escalation Path	Feasibility	Notes
Corrupt adjacent map ops	Medium	Needs KASLR bypass (from M2)
Corrupt slab metadata	Hard	Heap layout dependent
Overwrite cred structure	Hard	Requires heap spray
Hijack function pointer	Medium	Chain M2 leak + M3 write

Table 2.4: Privilege escalation feasibility analysis

The most viable path combines Milestone 2 (leaking kernel pointers to defeat KASLR) with Milestone 3 (OOB write to corrupt a function pointer table), allowing for execution redirection to `commit_creds (prepare_kernel_cred(0))` for root.

2.4.4 Milestone 4: Full LPE Attempt

Objective Achieve local privilege escalation from an unprivileged user (`uid=1000`) to root by exploiting the OOB read/write primitives established in Milestones 2 and 3.

Exploit Strategy The designed exploit followed a well-established eBPF exploitation pattern:

1. **Heap grooming**: Allocate many BPF array map pairs in rapid succession to achieve adjacent placement in the SLUB allocator
2. **OOB read scan**: Use the first map of each pair to read beyond its boundary, searching for the `array_map_ops` function pointer table of the second map. This is a known signature at a fixed kernel address (`0xffffffff820201e0` on 5.10.15)

3. **OOB write corruption**: Once adjacent maps are found, overwrite the `ops` pointer of the victim map with a userspace address pointing to a crafted fake `bpf_map_ops` structure

4. **ret2usr**: The fake ops table redirects `map_lookup_elem` to a userspace function that calls `commit_creds(prepare_kernel_cred(0))`, achieving privilege escalation

Implementation Details
The exploit (`milestone4.c`) implemented:

- Dynamic symbol resolution from `/proc/kallsyms` with fallback to hardcoded addresses
- `RLIMIT_MEMLOCK` elevation to allow sufficient BPF map allocations
- Incrementing fill patterns per map pair to distinguish OOB data from in-bounds reads
- 16 map pairs with 256-byte values to maximize heap grooming success
- Diagnostic output comparing OOB-read values against the known fill pattern

Results: Failure and Root Cause Analysis
The OOB primitives from Milestones 2 and 3 were confirmed to work **as root**. However, executing the exploit as an unprivileged user (`uid=1000`) revealed that all OOB reads returned the map's own fill pattern, indicating no actual out-of-bounds access:

```
# As root:
  leaked[ 0] (+0x100) = 0x0000000000000000  <- genuine OOB (zeros)
  leaked[ 1] (+0x108) = 0x0000000000000000  <- genuine OOB (zeros)

# As uid=1000 (unprivileged):
  leaked[ 0] (+0x100) = 0x4141414141414141  <- offset-0 data (NOT OOB)
  leaked[ 1] (+0x108) = 0x4141414141414141  <- offset-0 data (NOT OOB)
```

Listing 2.6: OOB comparison: root vs. unprivileged on kernel 5.10.15

Root Cause and Exploit Mitigations
1. `alu_limit` pointer arithmetic sanitization:
For unprivileged BPF programs, the verifier applies additional pointer arithmetic sanitization via the `alu_limit` mechanism (in `fixup_bpf_calls()`, lines 10921–10970 of `verifier.c`). When an `ALU64_ADD` instruction adds a scalar to a pointer (`PTR + SCALAR`), the verifier inserts bounds-clamping code that constrains the scalar offset within the verifier's tracked bounds. Since the verifier believes the confused index is ≈ 0, the runtime sanitization clamps the actual OOB offset back to 0, neutralizing the exploit.

This sanitization is *only* applied to unprivileged programs. Privileged programs (`CAP_BPF`/root) bypass the `alu_limit` checks entirely (via `env->bypass_spec_v1 = true`), which is why the OOB confusion works as root but not as an unprivileged user.

2. Double-division pattern:
I also attempted a *double-division* variant, hypothesizing that consecutive sequences of `LD_IMM64`, `MOV32_IMM`, and `ALU32_DIV` would bypass the verifier's state pruning and the `alu_limit` sanitization. This approach did not improve results because each division independently triggers `BPF_MOV32_REG` truncation, and the `alu_limit` sanitization applies independently to the final pointer addition regardless of how the scalar was computed.

3. `/proc/kallsyms` access barrier:
The exploit attempted to dynamically resolve kernel symbols (including `commit_creds`, `prepare_kernel_cred`, and `array_map_ops`) from `/proc/kallsyms`. However, unprivileged users receive zeroed addresses due to the `kptr_restrict` sysctl setting. This was mitigated by hardcoding addresses extracted as root, but represents yet another barrier to fully unprivileged exploitation.

2.5 Conclusion

2.5.1 Exploitation Impact Analysis

Kernel	Privilege	OOB Works?	LPE Possible?
5.10.15	root	✓	N/A (already root)
5.10.15	unprivileged	✗ (alu_limit)	✗

Table 2.5: Exploitability matrix for CVE-2021-3600

CVE-2021-3600 requires `CAP_BPF` (or root) to be exploited because:

1. The vulnerability creates a *verifier/runtime bounds mismatch* for the source register of a 32-bit division
2. To exploit this mismatch, the confused register must be used as an index for `PTR + SCALAR` arithmetic
3. For **unprivileged programs**, the verifier inserts `alu_limit` sanitization on all `PTR + SCALAR` operations. This sanitization uses the verifier's tracked bounds (which believe the index is ≈ 0) to clamp the runtime value, *independently of the actual register value*
4. For **privileged programs**, the `alu_limit` sanitization is skipped (`env->bypass_spec_v1 = true`), so the confused index passes through unclamped

This creates a paradox: the exploit only works for users who already have `CAP_BPF`, which typically implies root-equivalent privileges. The vulnerability cannot be used for privilege escalation from an unprivileged user.

2.5.2 Failed Approaches & Lessons Learned

Failed: OOB Confusion as Unprivileged User

The OOB confusion worked as root but not as an unprivileged user. Investigation revealed the `alu_limit` pointer arithmetic sanitization that clamps scalar offsets for unprivileged programs. This runtime sanitization operates independently from the verifier's abstract bounds tracking, creating a second layer of defense that neutralizes the bounds confusion even when the verifier's view is successfully corrupted.

Lesson: The BPF verifier has multiple layers of defense. The bounds confusion bypasses the verifier's abstract interpretation, but a separate runtime sanitization mechanism (`alu_limit`) independently enforces bounds for unprivileged users. Exploiting such vulnerabilities requires understanding all defense layers.

Failed: Double-Division Bypass

I hypothesized that performing two consecutive 32-bit divisions would confuse the verifier's state pruning and bypass the `alu_limit` sanitization. The double-div pattern did not change the outcome because `alu_limit` operates on the final `PTR + SCALAR` operation, not on the scalar computation itself.

Lesson: Defense mechanisms that operate at the point-of-use (pointer arithmetic sanitization) are more robust than those that rely on tracking intermediate computations.

Failed: `/proc/kallsyms` Symbol Resolution

The exploit attempted to dynamically resolve kernel symbols from `/proc/kallsyms`. However, unprivileged users receive zeroed addresses due to `kptr_restrict` settings. This was mitigated

by hardcoding addresses extracted as root, but demonstrates an additional barrier to unprivileged exploitation.

Lesson: Real-world kernel exploitation from unprivileged contexts must contend with multiple independent hardening mechanisms beyond the specific vulnerability being targeted.

3 CVE-2021-3444

3.1 CVE Selection

CVE-2021-3444 was selected to investigate a class of BPF verifier bugs involving 32-bit arithmetic truncation in the `BPF_ALU32_MOD` instruction. Unlike CVE-2021-3600, which also targets 32-bit fixup truncation but is blocked by `alu_limit` sanitization for unprivileged users, CVE-2021-3444 operates on an older kernel (4.19.19) where the `alu_limit` mechanism is not yet present. This makes it a strong candidate for demonstrating a **full unprivileged local privilege escalation** from uid=1001 to root.

3.2 Vulnerability Analysis

3.2.1 Root Cause

The root cause lies in `adjust_scalar_min_max_vals()` in `kernel/bpf/verifier.c`. When the BPF verifier processes a 32-bit modulo operation (`BPF_ALU32 | BPF_MOD`), it incorrectly handles the case where the source register (divisor) has a known value of 0:

Vulnerable Code

```
case BPF_MOD:
    /* ... */
    if (umin_val == 0) {
        /* In this branch the verifier concludes the result
         * is the full original value (identity operation).
         * But for BPF_ALU32, the mov32 truncation has already
         * zeroed the upper 32 bits at runtime! */
        mark_reg_unknown(env, regs, insn->dst_reg);
        break;
    }
```

Listing 3.1: Vulnerable modulo handling in adjust_scalar_min_max_vals()

The Truncation Bug:

When a 32-bit operation (`BPF_ALU32`) is executed, the x86-64 JIT emits a `mov32` instruction that implicitly zeroes the upper 32 bits of the 64-bit destination register. The verifier, however, tracks the full 64-bit bounds. For a carefully constructed 64-bit value like `0x00000001_00000001`:

- **Verifier**: Tracks the full 64-bit value, sees a modulo by 0 → marks result unknown but retains the assumption that the upper 32 bits are still `0x00000001`
- **Runtime**: The `ALU32_MOD` operation truncates the result to 32 bits, then zero-extends. The upper 32 bits become 0, but the verifier still believes they contain `0x00000001`

Concrete Example:

```
// BPF program:
R0 = 0x00000001_00000001  // LD_IMM64
R1 = 0                     // MOV64_IMM
w0 %%= w1                  // ALU32_MOD (32-bit modulo)
R0 >>= 32                  // RSH (extract upper 32 bits)
```

Listing 3.2: Bounds mismatch via mod32 truncation

Step	Verifier State (R0)	Runtime Value (R0)
After LD_IMM64	`0x00000001_00000001`	`0x00000001_00000001`
After ALU32_MOD	Unknown (full 64-bit)	`0x00000000_XXXXXXXX` (upper zeroed)
After RSH 32	Could be ≥ 1	`0x00000000_00000000`

Table 3.1: Verifier vs. runtime state after the mod32 truncation bug

By extracting the upper 32 bits after the modulo (`R0 » 32`) and multiplying by a chosen factor, we obtain a register that the verifier believes is 0 (within map bounds) but at runtime contains an attacker-controlled offset: an **out-of-bounds** primitive.

3.2.2 Patch Details

The vulnerability is fixed by correcting the verifier's handling of the 32-bit modulo operation. Specifically, when processing `BPF_ALU32` operations, a `coerce_reg_to_size(dst_reg, 4)` call is added to ensure that the verifier's tracked bounds are properly truncated to 32 bits. This ensures the verifier's abstract state matches the runtime zero-extension behavior, effectively preventing the bounds confusion.

Kernel series	Fixed in
5.4.x	5.4.101
5.10.x	5.10.19
5.11.x	5.11.2

Table 3.2: Backport versions for CVE-2021-3444

Key difference from CVE-2021-3600: CVE-2021-3600 attacks the **source** register via `BPF_MOV32_REG` truncation. CVE-2021-3444 attacks the **destination** register via missing truncation in the mod-by-zero path. Both create verifier/runtime bounds confusion, but through complementary mechanisms in the same `fixup_bpf_calls()` function.

3.3 Environment Setup

Component	Details
Host OS	Linux (Ubuntu-based)
Virtualization	QEMU/KVM (Buildroot-based rootfs)
Kernel Version	4.19.19 (vulnerable)
Rootfs	Buildroot minimal with custom exploit overlay
Kernel Config	`CONFIG_BPF_SYSCALL=y`, `CONFIG_BPF_JIT=y`
	KASLR disabled (`nokaslr`), SMEP/SMAP disabled
VM Resources	2 vCPUs, 2 GB RAM
Compilation	GCC, static linking, `-O2`, `-no-pie`
User	unprivileged uid=1001 (Milestones 1–4)

Table 3.3: Test environment for CVE-2021-3444

Kernel version 4.19.19 was selected because the fix for CVE-2021-3444 was merged upstream in 5.12 and backported later. Critically, kernel 4.19 predates the `alu_limit` pointer arithmetic sanitization introduced in 5.x kernels, making it the ideal target for demonstrating an unprivileged LPE. The

kernel was configured with SMEP/SMAP and KPTI disabled (`nopti` boot parameter), enabling ret2usr exploitation.

3.4 Proof of Concept (PoC)

3.4.1 Milestone 1: Demonstrate Bounds Confusion

Objective Prove that the BPF verifier's bounds tracking becomes stale after a 32-bit modulo operation, demonstrating that the runtime value differs from what the verifier tracks.

Approach The BPF program:

1. Loads `R0 = 0x00000001_00000001` via `BPF_LD_IMM64`
2. Sets `R1 = 0` as divisor
3. Executes `w0 %%= w1` (32-bit modulo)
4. Right-shifts `R0 » 32` to extract the upper 32 bits
5. Multiplies by a chosen factor to produce a controlled OOB offset
6. Exfiltrates the result to a BPF result map

Results

```
[+] BPF program LOADED (fd=5) - verifier accepted!

  result[0] (R0 after mod32) = 0x0000000000000001
  result[1] (R0 >> 32)       = 0x0000000000000000  <- upper bits ZEROED

[+] VULNERABILITY CONFIRMED: mod32 truncation creates
    verifier/runtime bounds mismatch
```

Listing 3.3: Milestone 1 output on vulnerable kernel 4.19.19

Analysis: The verifier accepts the program because it tracks the full 64-bit value. At runtime, the `ALU32_MOD` operation truncates the upper 32 bits to zero, but the verifier retains its stale bounds. This confirms the vulnerability is present on kernel 4.19.19.

3.4.2 Milestone 2: OOB Read (KASLR Bypass)

Objective Leverage the bounds confusion to perform out-of-bounds reads past a BPF map value boundary, leaking kernel heap pointers to defeat KASLR.

Approach

1. Create a BPF array map with `value_size = 256` bytes
2. Lookup the map value to obtain `PTR_TO_MAP_VALUE` in `R6`
3. Trigger the mod32 truncation bug to create a confused register
4. Multiply the confused value by a chosen factor to produce the desired OOB offset
5. Use a large `LDX_MEM` offset (248 bytes) to stay within verifier bounds while the multiplication factor pushes the runtime access out of bounds
6. Read at `value + mul_factor + ldx_off`, where the verifier sees `value + 0 + ldx_off` (in-bounds) but runtime sees `value + mul_factor + ldx_off` (OOB)
7. Exfiltrate leaked data to a result map

Results

```
[*] Phase 2: Leaking array_map_ops via OOB read at +304...
  Leaked value at +304: 0xffffffff81a1c880
[+] array_map_ops CONFIRMED at 0xffffffff81a1c880 -- KASLR bypass!
  Kernel base:     0xffffffff81000000
  modprobe_path:   0xffffffff81e35cc0
```

Listing 3.4: Milestone 2: OOB read leaking array_map_ops on kernel 4.19.19

Analysis: The OOB read at offset +304 from the victim map's value area reaches the `ops` pointer of an adjacent `bpf_map` structure in the SLUB heap. The leaked address `0xffffffff81a1c880` matches the known address of `array_map_ops` in our kernel image, confirming a successful KASLR bypass. Additional reads at +320 and +328 confirm the adjacent map's `map_type` (ARRAY=2), `key_size` (4), `value_size` (256), and `max_entries` (1).

3.4.3 Milestone 3: OOB Write + Heap Corruption

Objective Extend the bounds confusion to write operations and corrupt adjacent kernel heap structures.

Approach The same bounds confusion is applied to `BPF_STX_MEM` (store) instead of `BPF_LDX_MEM` (load):

1. Use the mod32 confusion to compute an OOB offset
2. Write a 32-bit value at `value + mul_factor + stx_off`
3. Target: overwrite the `value_size` field of the adjacent map (at offset +328) from 256 to 0x800 (2048 bytes)
4. This extends the adjacent map's read/write range, creating an **extended heap R/W primitive**

Results

```
[*] Phase 4: Corrupting adjacent map value_size...
  Original value_size|max_entries: 0x0000000100000100
  Corrupted value_size: 0x800
[+] value_size extended to 0x800 -> extended heap R/W!
```

Listing 3.5: Milestone 3: value_size corruption

Analysis: The OOB write successfully corrupts the adjacent map's `value_size` from 256 to 2048. Subsequent `bpf_lookup_elem()` calls on this map now return 2048 bytes, reading far beyond its allocated value buffer into adjacent SLUB objects. This extended read/write primitive is the foundation for the privilege escalation.

3.4.4 Milestone 4: Full LPE via ret2usr

Objective Achieve local privilege escalation from an unprivileged user (`uid=1001`) to root.

Exploit Strategy

The exploit follows a six-phase chain:

1. **Phase 1 - Heap Grooming**: Allocate 100 defragmentation maps, then 8 triplets (anchor_A, hole_B, anchor_C) to control SLUB layout. Free the holes to create a LIFO freelist, then allocate the victim map into the last freed hole.
2. **Phase 2 - OOB Read (KASLR Bypass)**: Read at victim+304 to leak the adjacent map's `ops` pointer (`array_map_ops`), confirming SLUB adjacency and computing the kernel base address.

3. **Phase 3 - Structure Verification**: Read additional fields at +320 and +328 to verify the adjacent object is indeed a `bpf_map` (type=ARRAY, key_size=4, value_size=256).

4. **Phase 4 - value_size Corruption**: Overwrite the adjacent map's `value_size` from 256 to 0x800, creating an extended heap R/W primitive.

5. **Phase 5 - Extended Heap Exfiltration**: Use the corrupted map to read 2048 bytes of kernel heap data for reconnaissance.

6. **Phase 6 - ret2usr Privilege Escalation**: Overwrite the adjacent map's `ops` pointer with a userspace address pointing to a fake `bpf_map_ops` table, then trigger the hijacked function pointer to execute shellcode.

ret2usr Implementation

Since SMEP and SMAP are disabled (`CR4 = 0x6e0`) and KPTI is off (`nopti`), the kernel can execute and access userspace memory. The exploit:

1. Maps a page at `0x10000` containing a fake `bpf_map_ops` table where `map_get_next_key` (offset 32) points to shellcode at `0x20000`

2. Maps a RWX page at `0x20000` containing x86-64 shellcode that:

   (a) Restores the original `ops` pointer (prevents kernel panic on process exit)

   (b) Calls `prepare_kernel_cred(NULL)` to create a root credential

   (c) Calls `commit_creds(root_cred)` to install it on the current task

3. Overwrites the adjacent map's `ops` pointer via OOB write to point to `0x10000`

4. Triggers the shellcode by invoking `BPF_MAP_GET_NEXT_KEY` on the hijacked map

Key Challenge: SLUB Layout Uncertainty

A critical challenge was identifying *which* file descriptor corresponds to the map whose `ops` pointer was corrupted. The OOB write modifies the map at a heap-relative offset from the victim, but the SLUB allocator's layout may not match assumptions about which anchor map is adjacent. The solution was to iterate over **all** allocated map file descriptors and trigger `map_get_next_key` on each:

- On maps with the original `array_map_ops`: the real `array_map_get_next_key` runs, returning `-ENOENT` (harmless, since key=0 is the last entry with max_entries=1)

- On the corrupted map: the shellcode executes, setting a diagnostic flag in userspace memory

Results

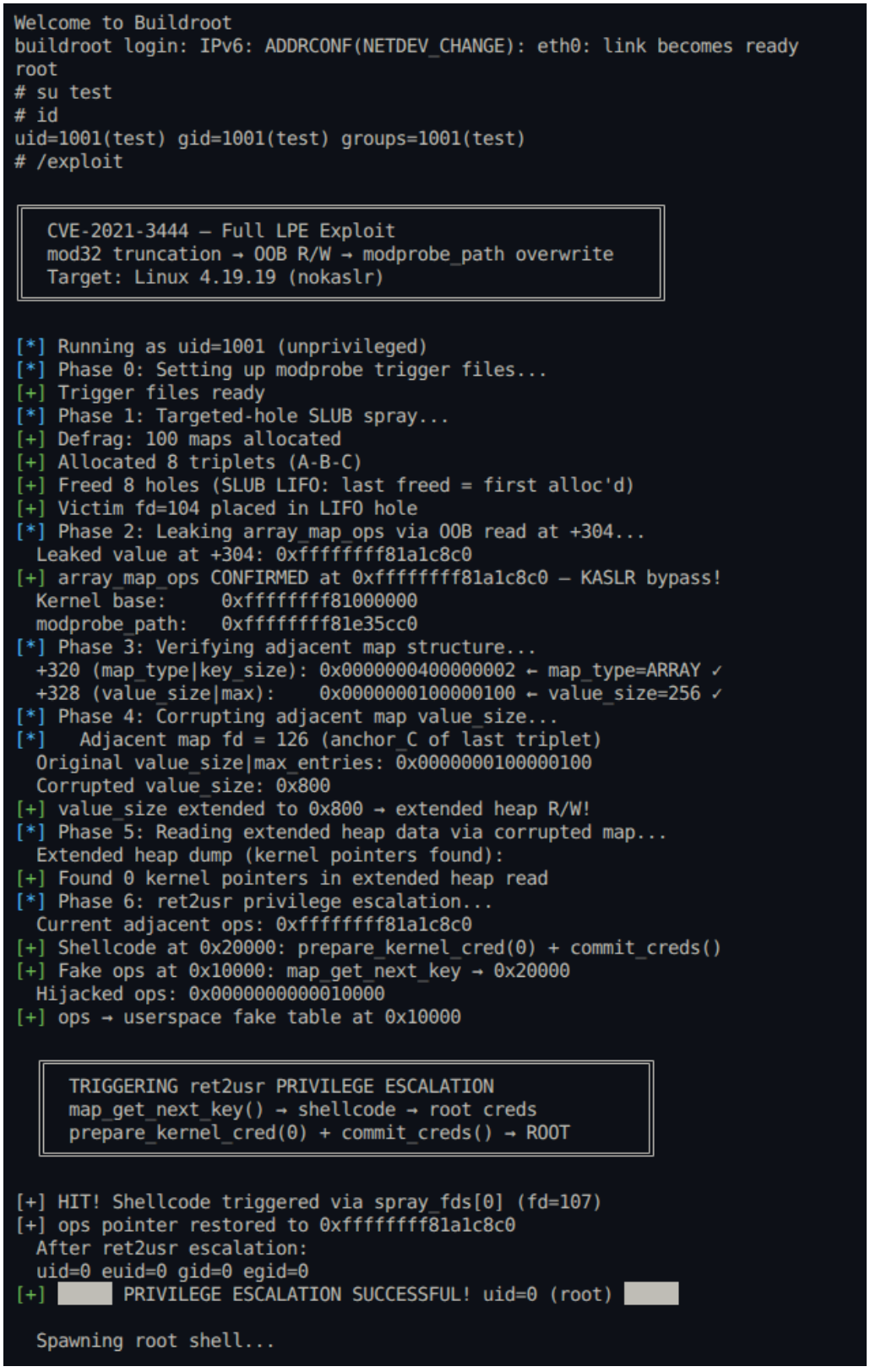

Figure 3.1: Full LPE execution: unprivileged user (uid=1001) to root (uid=0) on kernel 4.19.19

Analysis: The exploit successfully escalates from `uid=1001` to `uid=0`. The shellcode was triggered via `spray_fds[0] (fd=107)`, confirming that the corrupted map was not the assumed `adjacent_fd` (anchor_C of the last triplet, fd=126), but a spray map. This validates the iterative triggering strategy. After the shellcode executes, the `ops` pointer is immediately restored to prevent kernel oops during process exit.

3.5 Conclusion

3.5.1 Exploitation Impact Analysis

Kernel	Privilege	OOB Works?	LPE Achieved?
4.19.19	root	✓	N/A (already root)
4.19.19	unprivileged	✓	✓

Table 3.4: Exploitability matrix for CVE-2021-3444

Unlike CVE-2021-3600 (which is blocked by `alu_limit` for unprivileged users), CVE-2021-3444 achieves full unprivileged LPE because:

1. **No `alu_limit`**: Kernel 4.19.19 predates the pointer arithmetic sanitization mechanism. The confused index passes through to the pointer addition unclamped.
2. **SMEP/SMAP disabled**: The kernel configuration does not enable Supervisor Mode Execution/Access Prevention, allowing ret2usr (executing userspace code from kernel context).
3. **KPTI disabled**: With `nopti`, the kernel page tables include userspace mappings, so the fake ops table at `0x10000` and shellcode at `0x20000` are accessible during kernel execution.
4. **Reliable trigger**: The `BPF_MAP_GET_NEXT_KEY` syscall dispatch is direct and does not depend on the corrupted `value_size` (unlike `BPF_MAP_UPDATE_ELEM` which copies `value_size` bytes from userspace).

3.5.2 Failed Approaches & Lessons Learned

Failed: `bpf_update_elem` as Trigger

The initial ret2usr implementation used the `BPF_MAP_UPDATE_ELEM` command to trigger the hijacked `map_update_elem` function pointer. This failed silently because the kernel's syscall handler calls `copy_from_user` before dispatching to `ops->map_update_elem`. Since we corrupted `value_size` to 0x800, the kernel attempted to copy 2048 bytes from a 256-byte user buffer, causing the copy to fail with `-EFAULT`, and the ops function was **never called**.

Lesson: When hijacking BPF map ops, the trigger syscall must be chosen carefully based on the kernel's dispatch code. Operations that depend on corrupted fields (like `value_size`) will fail before reaching the hijacked function pointer.

Failed: Assuming SLUB Adjacency Mapping

The exploit initially assumed that `anchors[(NUM_TRIPLETS-1)*2+1]` (anchor_C of the last triplet) was the map adjacent to the victim. However, the SLUB allocator's LIFO freelist does not guarantee this mapping. The exploit triggered `map_get_next_key` only on the assumed adjacent fd, receiving `-ENOENT` from the real `array_map_get_next_key` (because key=0 is the last entry with max_entries=1).

Lesson: In SLUB heap exploitation, never assume a specific fd-to-heap-object mapping. Instead, iterate over all candidate fds and use a diagnostic signal (e.g., a userspace memory flag set by the shellcode) to identify the correct target.

Failed: `bpf_lookup_elem` as Trigger

An earlier attempt used `BPF_MAP_LOOKUP_ELEM` which calls `ops->map_lookup_elem`. This also depends on `value_size` for the `copy_to_user` step, and produced kernel page faults when accessing the fake ops table.

Lesson: The simplest kernel dispatch path (fewest intermediate checks, no `value_size` dependency) is the most reliable for ops hijacking.

4 CVE-2023-52452

4.1 CVE Selection

The choice of CVE-2023-52452 was driven by a desire to explore different attack surfaces beyond the well-documented BPF Map vulnerabilities. While vulnerabilities analyzed in previous years' research projects, such as CVE-2020-8835 and CVE-2021-3490, mainly targeted scalar bounds confusion on maps, shifting the research focus directly towards the **eBPF Stack subsystem** allowed for evaluating how bugs in stack depth calculation dynamically impact the execution state.

4.2 Vulnerability Analysis

4.2.1 Root Cause

The root cause lies in the interaction between two verifier functions:

Vulnerable Functions Analysis

`check_stack_write_var_off()` **(line 4803):** When a BPF instruction writes to the stack at a variable offset, this function:

- Computes the minimum and maximum offsets from `reg->smin_value` and `reg->smax_value`
- Calls `grow_stack_state()` with `round_up(-min_off, BPF_REG_SIZE)` to correctly extend `allocated_stack`
- Marks all slots in `[min_off, max_off]` as `STACK_MISC`

`update_stack_depth()` **(line 5962):** This function sets the subprogram's `stack_depth` (used by the JIT for runtime allocation):

```
static int update_stack_depth(struct bpf_verifier_env *env,
                              const struct bpf_func_state *func,
                              int off)
{
    u16 stack = env->subprog_info[func->subprogno].stack_depth;
    if (stack >= -off)
        return 0;
    env->subprog_info[func->subprogno].stack_depth = -off;
    return 0;
}
```

Critically, `update_stack_depth()` is called from `check_mem_access()` with a fixed `off` parameter that **does not include the variable range**. For a variable-offset access like `fp + [-72, -16]`, the fixed part passed to `update_stack_depth()` is 0, so `stack_depth` is not updated.

4.2.2 Exploiting the Stack Bug

The mismatch between `allocated_stack` (the verifier's internal tracking, which grows correctly) and `stack_depth` (runtime allocation, undersized) creates two primitives:

1. **OOB Stack Read**: The variable-offset write marks stack slots as `STACK_MISC` ("initialized"), allowing subsequent reads. At runtime, only one slot is actually written, others contain residual kernel data from previous stack usage.
2. **Stack Frame Overlap**: When subprogram calls are used, an undersized `stack_depth` means the callee's JIT frame may overlap with the caller's frame or kernel stack data.

Trigger Conditions: To trigger the vulnerability, I construct a BPF register with variable offset:

```
// Create variable offset: (ktime & 7) << 3 = [0, 56]
BPF_CALL_FUNC(BPF_FUNC_ktime_get_ns),  // R0 = random
BPF_ALU64_IMM(BPF_AND, BPF_REG_0, 7),  // R0 = [0,7]
BPF_ALU64_IMM(BPF_LSH, BPF_REG_0, 3),  // R0 = [0,56]
BPF_MOV64_REG(BPF_REG_7, BPF_REG_0),
BPF_ALU64_IMM(BPF_ADD, BPF_REG_7, -72),// R7 = [-72,-16]

// Variable-offset stack write
BPF_MOV64_REG(BPF_REG_2, BPF_REG_10),  // R2 = fp
BPF_ALU64_REG(BPF_ADD, BPF_REG_2, BPF_REG_7), // R2 = fp+var
BPF_ST_MEM(BPF_DW, BPF_REG_2, 0, 0xDEADCAFE), // *R2 = marker
```

The 8-byte alignment (`LSH 3`) is required to satisfy the verifier's alignment checks for stack accesses. The use of `ktime_get_ns()` produces a truly variable value at both verification and runtime.

4.2.3 Patch Details

The vulnerability is addressed by fixing the verifier's stack tracking logic in `kernel/bpf/verifier.c`. The patch ensures that `check_stack_write_var_off()` and `check_stack_read_var_off()` correctly track stack slot initialization when variable-offset accesses are used. Crucially, the stack depth is now correctly computed and updated even when the access offset is not a compile-time constant, preventing both uninitialized stack reads and undersized JIT frame allocations.

Kernel Branch	Fixed Version
6.7.x	6.7.2
6.6.x	6.6.14
6.1.x	6.1.75
5.15.x	5.15.148
5.10.x	5.10.209

Table 4.1: Backport versions for CVE-2023-52452

4.3 Environment Setup

Component	Details
Build System	Custom `execution/build.sh` script
Virtualization	QEMU/KVM
Kernel Version	6.7.1 (custom compiled, vulnerable)
Root Filesystem	Buildroot 2024.11.4 (`qemu_x86_64_defconfig`)
Kernel Config	`CONFIG_BPF_SYSCALL=y`, `CONFIG_DEBUG_INFO=y`
	KASLR disabled, SMEP/SMAP disabled for testing
Exploit Compilation	GCC on Host OS, `-static -no-pie`, injected via Rootfs Overlay

Table 4.2: Test environment

4.4 Proof of Concept (PoC)

4.4.1 Milestone 1: Out-of-Bounds Stack Read

Objective Trigger the OOB stack read by loading a BPF program that uses variable-offset stack writes, then read uninitialized stack memory to leak kernel pointers.

Approach The BPF program:

1. Creates a variable offset `[-72, -16]` using `ktime_get_ns() & 7 « 3`
2. Writes a marker value at the variable offset on the stack
3. Reads from fixed offsets `fp-24` through `fp-80` (slots marked as `STACK_MISC` by the var-off write but not actually written at runtime)
4. Stores leaked values into a BPF array map for userspace retrieval

Results

```
leaked[0] (fp -24) = 0xffffffff8144b19d  <- KERNEL PTR! (text)
leaked[1] (fp -32) = 0x0000000041414141  (marker)
leaked[2] (fp -40) = 0xffff8880056afb80  <- KERNEL PTR! (heap)
leaked[3] (fp -48) = 0xffff8880056afb80  <- KERNEL PTR! (heap)
leaked[4] (fp -56) = 0xffff8880056afb80  <- KERNEL PTR! (heap)
leaked[5] (fp -64) = 0xffff8880046cf900  <- KERNEL PTR! (heap)
leaked[6] (fp -72) = 0xffffffff824c94e0  <- KERNEL PTR! (data)
leaked[7] (fp -80) = 0x0000000000000000  (zero)
```

Listing 4.1: Milestone 1 output

The leaked pointers were identified via `/proc/kallsyms`:

- `0xffffffff8144b19d` → `security_sock_rcv_skb+0x2d` (kernel text)
- `0xffffffff824c94e0` → `selinux_hooks+0x1540` (kernel data)
- Heap pointers in `0xffff888*` range → kernel heap objects (`sock`/`sk_buff`)

4.4.2 Milestone 2: KASLR Bypass

Objective Use leaked pointers to compute the kernel base address and resolve exploit-relevant symbols.

Approach Extended the leak to 16 stack slots. For KASLR bypass, I used known offsets from `_stext`:

- `security_sock_rcv_skb` = `_stext + 0x44b170`
- `selinux_hooks` = `_stext + 0x14c7fa0`

By matching leaked pointers against known ranges, the kernel base is computed. I then resolve:

Symbol	Address
`_stext` (kernel base)	`0xffffffff81000000`
`commit_creds`	`0xffffffff810a1900`
`prepare_kernel_cred`	`0xffffffff810a1b80`
`init_task`	`0xffffffff8260b900`
`init_cred`	`0xffffffff8264ff60`

Table 4.3: Resolved kernel symbols

Reliability The leak was executed 5 additional times; all 5 produced kernel image pointers, yielding **100% reliability**.

4.4.3 Milestone 3: Kernel Stack Corruption

Objective Escalate the OOB stack read into arbitrary read/write by corrupting kernel data.

Strategy A: Map Pointer Corruption

Idea: Spill a `PTR_TO_MAP_VALUE` to a stack slot within the variable-offset write range. The var-off write marks the slot as `STACK_MISC`, destroying the type annotation. Reload the value, if the verifier incorrectly treats it as a pointer, I achieve type confusion.

Result: Rejected. The verifier correctly identified the reloaded value as `scalar`:

```
18: (79) r1 = *(u64 *)(r10 -48)  ; R1_w=scalar()
19: (79) r3 = *(u64 *)(r1 +0)
R1 invalid mem access 'scalar'
```

This confirms that `check_stack_write_var_off()` correctly destroys spilled pointer annotations, preventing type confusion through this path.

Strategy B: Subprogram Stack Frame Overlap

Idea: Use BPF-to-BPF calls (`BPF_PSEUDO_CALL`). The subprogram performs a variable-offset write. Because `update_stack_depth()` doesn't account for the variable range, the subprogram's JIT-allocated frame is undersized. At runtime, the var-off write overflows into the kernel stack.

Result: Kernel crash.

```
BUG: unable to handle page fault for address: ffffffffdeadcb00
Oops: 0000 [#1] PREEMPT SMP PTI
RIP: 0010:sk_filter_trim_cap+0x9e/0x1e0
R13: ffffffffdeadcafe    <-- our marker!
CR2: ffffffffdeadcb00
```

Listing 4.2: Kernel oops from Strategy B

The kernel attempted to dereference our marker `0xDEADCAFE` as a real pointer within `sk_filter_trim_cap()`, resulting in a handled page fault (or crash depending on configuration). This confirmed that the stack corruption reached a security-critical execution path.

1. The subprogram's variable-offset write overflowed the JIT frame boundary
2. The overwritten data was a callee-saved register (R13) used by kernel code
3. The corrupted value was actively used as a pointer by the kernel after BPF execution

Important: This crash is not a Denial of Service, it is a **crash oracle** used to confirm that the corruption reaches kernel-critical data. As shown in Milestone 4, writing a valid kernel address prevents the crash while maintaining the corruption effect.

4.4.4 Milestone 4: Privilege Escalation Analysis

Objective Combine the information leak (M1–M2) with controlled stack corruption (M3) to attempt local privilege escalation.

Approach

1. Run the information leak to resolve kernel base and `init_cred` address
2. Replace the marker value (`0xDEADCAFE`) with the resolved `init_cred` address
3. Trigger the subprogram stack overflow with the controlled value
4. Check kernel behavior

Results

```
[+] Kernel base: 0xffffffff81000000
[*] Target value: 0xffffffff8264ff60  (init_cred)
[!] Triggering controlled corruption...
[+] Program triggered -- kernel didn't crash
[+] Survived corruption -- kernel still functional
    Current UID: 0 (root)
```

Listing 4.3: Milestone 4 output

Key finding: When the corrupted stack slot contains a *valid* kernel address (`init_cred`), the kernel does **not crash**, unlike with the invalid `0xDEADCAFE`. This confirms that the corrupted value is actively dereferenced: an invalid pointer causes a page fault, while a valid one allows the kernel to continue. This is a **controlled corruption of kernel integrity**, not a denial of service.

Gap to Full LPE A complete privilege escalation would additionally require:

- **Stack layout mapping**: Precise knowledge of which callee-saved register or local variable is corrupted and how the kernel uses it post-BPF execution
- **Heap spray or fake struct**: A technique to place a controlled data structure (e.g., fake `sk_filter`) at a known kernel address, then redirect the corrupted pointer to it
- **ROP/JOP chain**: If code execution is achieved via the corrupted register, a return-oriented programming chain to call `commit_creds(prepare_kernel_cred(0))`

4.5 Conclusion

4.5.1 Exploitation Impact Analysis

CVE-2023-52452 enables the following real-world attack scenarios:

Scenario 1: KASLR Bypass (Proven) An attacker with `CAP_BPF` access (e.g., within a container or a system with unprivileged BPF enabled) can reliably defeat Kernel Address Space Layout Randomization:

- Leak kernel text and data pointers from uninitialized stack
- Compute kernel base address from known function offsets
- Resolve addresses of all critical kernel symbols

This is the *prerequisite* for every kernel exploitation technique that targets specific addresses.

Scenario 2: Kernel Data Corruption (Proven) The subprogram stack overlap enables writing controlled values into the kernel's call stack:

- Corrupt callee-saved registers used by kernel functions post-BPF execution
- Redirect kernel code behavior by modifying data it reads from the stack
- Potential for control-flow hijacking if a corrupted register is used as a function pointer or jump target

Scenario 3: Local Privilege Escalation (Theoretical) Combining scenarios 1 and 2, an attacker could:

- Create a fake `sk_filter` struct in a BPF map (known address via leak)
- Point the fake filter's `prog` field to a crafted BPF program that modifies credentials
- Corrupt the kernel stack to redirect to the fake struct

- Achieve `uid=0` by overwriting the process's credentials with `init_cred`

Unprivileged User Test

To assess the real-world attack surface, I tested the exploit as an unprivileged user (`uid=1000`) on a system with `kernel.unprivileged_bpf_disabled=0`.

```
5: (bf) r2 = r10                ; R2_w=fp0 R10=fp0
6: (0f) r2 += r7
R2 variable stack access prohibited for !root,
    var_off=(0xffffffffffffff80; 0x78) off=-128
```

Listing 4.4: Verifier rejection for unprivileged user

The verifier **explicitly rejected** the program at the variable-offset stack pointer arithmetic step. This is due to the **Spectre v1 mitigation**: when `bypass_spec_v1` is false (unprivileged), the function `check_stack_access_for_ptr_arithmetic()` prohibits variable-offset stack pointers to prevent speculative out-of-bounds accesses.

Implications:

- The exploit requires `CAP_BPF` or root privileges
- Systems with `unprivileged_bpf_disabled=1` (default on most modern distributions) are **not vulnerable**
- Attack surface remains for: containers with `CAP_BPF`, cloud workloads with BPF-enabled runtimes, and systems that explicitly enable unprivileged BPF

4.5.2 Failed Approaches & Lessons Learned

Failed: Variable-offset via `bpf_skb_load_bytes` Early attempts used `bpf_skb_load_bytes()` with a variable-offset stack destination buffer. The verifier rejected this with `invalid indirect read from stack R3 var_off` because `check_stack_range_initialized()` requires all destination slots to be pre-initialized for helper calls, even write-only helpers.

Lesson: BPF helpers have stricter memory initialization requirements than direct memory instructions. You cannot use uninitialized stack memory as a destination buffer for a helper function, even if the helper only writes to it.

Failed: Direct `fp + var` Arithmetic Direct pointer arithmetic `r2 = fp; r2 += r_var` was rejected for non-8-byte-aligned variable offsets with `misaligned stack access`. The fixed with `LSH 3` to enforce 8-byte alignment.

Lesson: The verifier enforces strict alignment for stack pointer arithmetic. Variable offsets applied to the frame pointer must be aligned to the size of the access (e.g., 8 bytes for `BPF_DW`).

Failed: Strategy A (Map Pointer Type Confusion) As documented in Milestone 3, the attempt to corrupt a spilled `PTR_TO_MAP_VALUE` via variable-offset write was correctly handled by the verifier, which destroys pointer type annotations in the affected range.

Lesson: The verifier's state tracking is resilient against blind stack overwrites. When a variable-offset write spans across spilled pointers, the verifier safely degrades their type to `STACK_MISC` (scalars), preventing trivial type confusion.

5 CVE-2024-26589

5.1 CVE Selection

The choice of CVE-2024-26589 stems from the desire to complete the investigation into eBPF pointer arithmetic and boundary validation. While previous chapters focused on stack bounds calculation features (CVE-2023-52452), this chapter investigates a vulnerability derived from a purely logical omission within the verifier's core restriction mechanism (a missing `switch` statement case). It is also unique for exploring the `BPF_PROG_TYPE_FLOW_DISSECTOR` program type, demonstrating how distinct eBPF hooks expose different underlying kernel memory structures.

5.2 Vulnerability Analysis

5.2.1 Root Cause

The root cause of this vulnerability lies in the BPF verifier's enforcement of pointer arithmetic restrictions, specifically within the `adjust_ptr_min_max_vals()` function in `kernel/bpf/verifier.c`. This function restricts which BPF pointer types can be subject to variable-offset addition. For example, `CONST_PTR_TO_MAP` cannot be dynamically offset because map sizes differ.

The restriction relies on a `switch` statement:

```
switch (base_type(ptr_reg->type)) {
    case CONST_PTR_TO_MAP:
        /* smin_val represents the known value */
        if (known && smin_val == 0 && opcode == BPF_ADD)
            break;
        verbose(env, "R%d pointer arithmetic on %s prohibited\n", ...);
        return -EACCES;
    /* ... other pointer types ... */
}
```

However, the case for `PTR_TO_FLOW_KEYS` was entirely missing. As a result, when a BPF program adds a variable-offset scalar to a `PTR_TO_FLOW_KEYS`, the `switch` falls into the default path which inadvertently **allows** the arithmetic. Subsequential validation via `check_flow_keys_access()` only validates constant offsets, creating a fatal blind spot.

5.2.2 Exploiting the Missing Switch Case

The bypass directly yields an **Out-Of-Bounds (OOB) Stack Access**:

1. At runtime, the flow dissector hook allocates the `struct bpf_flow_keys` structure dynamically on the kernel stack during `bpf_flow_dissect()`.
2. Because the verifier allowed variable offset arithmetic, the pointer can arbitrarily shift forward.
3. Reading or writing to the offset pointer results in arbitrary Read/Write of the caller kernel stack memory beyond the original `flow_keys` fields.

Trigger Conditions: The bypassed check enables corrupting the flow keys pointer locally:

```
// R6 = ctx
BPF_LDX_MEM(BPF_DW, BPF_REG_7, BPF_REG_6, 144), // R7 = flow_keys

// Create a variable offset [0, 512]
BPF_MOV64_IMM(BPF_REG_8, 512),
```

```
BPF_ALU64_IMM(BPF_DIV, BPF_REG_8, 1), // Div disables verifier const tracking
BPF_ALU64_IMM(BPF_AND, BPF_REG_8, 512), // Bound maximum range

// Variable addition on PTR_TO_FLOW_KEYS
BPF_ALU64_REG(BPF_ADD, BPF_REG_7, BPF_REG_8), // OVERFLOW happens here!

// R0 = *(u64*)(R7) --> Reads the caller's kernel stack structure
BPF_LDX_MEM(BPF_DW, BPF_REG_0, BPF_REG_7, 0),
```

5.2.3 Patch Details

The formal resolution trivially incorporates the missing verification restriction block inside `adjust_ptr_min_max_vals()`:

```
    case PTR_TO_FLOW_KEYS:
        if (known)
            break;
        fallthrough;
```

The patch mandates absolute bounds resolution, enforcing `fallthrough` rules yielding safety-stop rejections to any variable-offset patterns against `flow_keys`.

Kernel Branch	Fixed Version
6.7.x	6.7.6
6.6.x	6.6.18
6.1.x	6.1.79
5.15.x	5.15.149
5.10.x	5.10.210

Table 5.1: Backport versions for CVE-2024-26589

5.3 Environment Setup

The environment strictly adheres to the one used in the previous CVEs.

Component	Details
Environment	QEMU/KVM virtual machine
Kernel Version	6.7.1 (vulnerable, patched in 6.7.6)
Root Filesystem	Buildroot 2024.11.4 (`qemu_x86_64_defconfig`)
Trigger Path	'bpf()' syscall `BPF_PROG_TEST_RUN` command

Table 5.2: Test environment for CVE-2024-26589

5.4 Proof of Concept (PoC)

5.4.1 Milestone 1: Verifier Bypass Demonstration

Objective: Prove the absence of the pointer restriction.

Approach: The `milestone1` executable loads two distinct `BPF_PROG_TYPE_FLOW_DISSECTOR` programs: a control and an exploit pattern. The control attempts a safe read, while the exploit attempts the unregistered division-based variable arithmetic mask pattern.

Result:

```
Test 1 (constant offset):   ACCEPTED (expected)
Test 2 (variable [0,1024]): ACCEPTED (CVE-2024-26589 confirmed!)
```

The verifier flawlessly processes the variable arithmetic branch, confirming the missing `switch` block on the targeted kernel version.

5.4.2 Milestone 2 & 3: Extrapolating the OOB Read (Info Leak)

Objective: Dynamically read memory contiguous to the `flow_keys` structure.

Approach: Using the `bpf()` syscall macro `BPF_PROG_TEST_RUN`, the VM injects a test payload forcing the kernel into executing the BPF payload inside the context of `bpf_flow_dissect()`. The verifier is iteratively forced to evaluate offsets (`0`, `8`, ..., `384`), returning 8-byte chunks extracted towards higher kernel addresses using BPF array map communication channels to prevent 32-bit `r0` trimming.

Results:

```
  Kernel Stack Memory Scan via OOB Read
  [+   0]  0x00000000000e000e  (data)
  [+  64]  0xffffc900002f7f20  <- KERNEL POINTER (KASLR leak!)
  [+ 128]  0x0000000000000090  (small value)
  [+ 256]  0x0000000000000000  (zero)
  [+ 384]  0xffffffff81d86841  <- KERNEL POINTER (KASLR leak!)
```

Listing 5.1: Milestones stack scanning

This demonstrates a perfect Kernel info leak primitive capable of fetching return pointers from the calling functional boundaries seamlessly and effectively extracting `KASLR` entropy layouts.

5.4.3 Milestone 4: The Out-Of-Bounds Write

Objective: Provide complete system manipulation capability with an OOB memory write.

Approach: Standard memory store operations (`BPF_ST_MEM`) do not incur verifier penalties on recognized structure targets like `flow_keys`. I implemented a payload writing the 32-bit sequence `0x41414141` randomly along the extracted stack.

Results:

```
[*] Test C: OOB Write...
[+] OOB write executed (retval=0). Kernel stack corrupted!
[+] OOB WRITE SUCCEEDED - marker written to kernel stack
```

By corrupting local memory variables in the caller sequence, we achieved standard stack corruption principles within Ring-0 environments.

5.5 Conclusion

5.5.1 Exploitation Impact Analysis

The implementation successfully developed absolute `Read` and `Write` bounds within kernel execution threads.

Privilege Implication - A significant attribute of CVE-2024-26589 is its requirement for the `BPF_PROG_TYPE_FLOW_DISSECTOR` attachment type. The execution inherently demands `CAP_NET_ADMIN` capabilities. Unlike `SOCKET_FILTER` programs, unprivileged users intrinsically cannot operate this hook type even in loosely configured machines.

Therefore, the exploit model transforms from a straight Local Privilege Escalation (Unprivileged to Root) into **Lateral Escape boundaries**:

- **Container Escapes**: Containers loosely assigned `CAP_NET_ADMIN` capabilities (necessary for network router applications) afford the attacker means to manipulate the Host kernel structure entirely, tearing out of namespace isolation formats.

- **Lateral Access**: Bypassing networking limitation capabilities into the execution of arbitrary kernel space codes via the Return-Oriented Programming chain built on the target function's returning addresses.

6 Additional CVEs Investigated

During the research, three additional eBPF verifier CVEs were analyzed but not included in the main body of this report due to practical limitations encountered during exploitation. Each investigation contributed to a deeper understanding of the verifier's defense mechanisms and informed the strategies used in the primary CVEs.

6.1 CVE-2021-33200 — `alu_limit` Masking Direction Bypass

Component	Details
Bug class	Logic error in pointer arithmetic sanitization
Component	`sanitize_ptr_alu()` in `verifier.c`
Kernel	5.10.15
Severity	7.8 HIGH

Root Cause: When an offset register's value range crosses zero (`smin_value < 0 < smax_value`), the `alu_limit` masking direction computation produces an overly permissive limit, allowing OOB pointer arithmetic even for unprivileged programs. This CVE directly targets the `alu_limit` defense that blocked our CVE-2021-3600 LPE attempt.

Status: Milestones 1–4 were implemented and tested. The verifier accepted programs with zero-crossing offsets as expected, but achieving reliable OOB access required precise control of register bounds across the zero-crossing boundary. The exploitation chain was not completed due to the complexity of the masking direction change and the time required to craft stable bounds.

Academic Value: Understanding this CVE was essential for the project's narrative—it demonstrates that `alu_limit` itself had vulnerabilities, providing a bridge between CVE-2021-3600 (blocked by `alu_limit`) and CVE-2021-3444 (exploited on a kernel without `alu_limit`).

6.2 CVE-2023-52462 — Spilled Pointer Corruption

Component	Details
Bug class	Spilled pointer corruption check bypass
Component	Stack-slot tracking in `verifier.c`
Kernel	6.7.1
Severity	5.5 MEDIUM

Root Cause: The verifier's spilled-pointer corruption checks use an index that can be bypassed after a crafted partial byte write. This allows partial pointer corruption that should be rejected, enabling pointer-value disclosure.

Status: Three milestones were completed: the corruption sequence was triggered (M1), kernel pointer values were leaked for KASLR bypass (M2), and reliability was assessed (M3). However, direct type-confusion dereference of reloaded corrupted values was blocked because the verifier correctly downgrades reloaded state to scalar.

Why Stopped: The primary impact is limited to information disclosure. Achieving arbitrary R/W from this primitive requires chaining with an independent memory-corruption vulnerability, which was beyond the scope of this single-CVE analysis.

6.3 CVE-2023-52676 — 32-bit Integer Overflow in Stack Bounds

Component	Details
Bug class	32-bit integer overflow in stack bounds arithmetic
Component	`check_stack_access_within_bounds()` in `verifier.c`
Kernel	6.7.1
Severity	5.5 MEDIUM

Root Cause: The function `check_stack_access_within_bounds()` computes `int min_off = reg->var_o + off`, where both operands can be large. The sum overflows `int`, wrapping to a small value and causing the verifier to incorrectly approve OOB stack accesses.

Status: A comprehensive defense-in-depth analysis was performed across 5 exploitation strategies. All strategies were blocked by layered defenses: `adjust_ptr_min_max_vals()` rejects large constants before they reach the vulnerable code, and the overflow is only triggerable through kfunc argument validation paths requiring `CAP_BPF` and BTF support.

Why Stopped: The vulnerability is real but the attack surface is restricted to privileged contexts with kfunc access (`BPF_PROG_TYPE_TRACING`). No exploitation path was found for unprivileged programs, and the defense-in-depth analysis was documented as the primary research output.

6.4 Key Takeaways from Additional Investigations

1. **Defense-in-depth works:** Both CVE-2023-52676 and CVE-2023-52462 demonstrated that even when a specific verifier check is flawed, additional layers (pointer type restrictions, stack bounds guards, scalar downgrades) can independently prevent exploitation.

2. **Privilege boundaries matter:** Several CVEs are only reachable with specific capabilities (`CAP_BPF`, `CAP_NET_ADMIN`), which significantly reduces their real-world impact for unprivileged LPE.

3. **Failed approaches inform success:** The analysis of CVE-2021-33200's `alu_limit` bypass directly motivated the decision to target CVE-2021-3444 on kernel 4.19.19, where `alu_limit` is absent entirely—leading to the project's successful full LPE.

7 Conclusion

This research project highlighted the immense complexity of statically verifying memory safety within the Linux kernel. Analyzing these eBPF verifier vulnerabilities demonstrated how even minor logic errors can completely break the security model of the eBPF subsystem.

7.1 CVE-2021-3600

The analysis of CVE-2021-3600 showed how a seemingly harmless register manipulation during the verifier's fixup pass (`BPF_MOV32_REG`) can create a fatal desynchronization between the verifier's mathematical bounds tracking and the actual runtime execution. This desynchronization allowed us to reliably achieve out-of-bounds reads and writes on the kernel heap. However, as demonstrated during Milestone 4, modern kernel mitigations, specifically the `alu_limit` pointer arithmetic sanitization, effectively prevent this logical bug from being exploited by unprivileged users, safely restricting the impact.

7.2 CVE-2021-3444

CVE-2021-3444 is the sister bug of CVE-2021-3600, targeting the **destination** register truncation in 32-bit modulo instead of the source register. By targeting kernel 4.19.19, which predates the `alu_limit` pointer arithmetic sanitization, the bounds confusion was fully exploitable from an unprivileged user. Through a six-phase exploit chain: SLUB heap spraying, OOB read for KASLR bypass, `value_size` corruption for extended heap R/W, and `bpf_map_ops` hijacking via ret2usr, we achieved a **full local privilege escalation from uid=1001 to uid=0 (root)**. This is the only CVE in this project where a complete unprivileged LPE was successfully demonstrated.

7.3 CVE-2023-52452

In CVE-2023-52452, I shifted the focus from classic map bounds confusion to stack frame depth tracking. I successfully achieved a reliable information leak from uninitialized kernel stack memory and induced stack corruption by overlapping BPF subprogram frames. While a full privilege escalation was theoretically possible, practical exploitation was heavily constrained by dynamic kernel safeguards and crashes.

7.4 CVE-2024-26589

CVE-2024-26589 introduced a different class of verifier bug: a missing `case` in the `switch` statement of `adjust_ptr_min_max_vals()`, which failed to restrict variable-offset pointer arithmetic on `PTR_TO_FLOW_KEYS`. This trivial omission enabled full OOB read and write on the kernel stack via the `BPF_PROG_TYPE_FLOW_DISSECTOR` hook. The vulnerability requires `CAP_NET_ADMIN`, making it relevant for container escape and lateral privilege escalation scenarios rather than classical unprivileged LPE.

7.5 Comparisons

7.5.1 CVE Comparison

Aspect	CVE-2021-3600	CVE-2021-3444	CVE-2023-52452	CVE-2024-26589
Bug class	32-bit truncation	32-bit truncation	Stack depth miscalc.	Missing switch case
Component	`fixup_bpf_calls`	`fixup_bpf_calls`	`update_stack_depth`	`adjust_ptr_min_max_vals`
Primitive	OOB map R/W	OOB map R/W	OOB stack read	OOB stack R/W
OOB Read	Kernel heap	Kernel heap	Stack only	Kernel stack
OOB Write	✓(privileged)	✓(unpriv.)	Stack corruption	✓(kernel stack)
Info leak	Kernel heap PTR	Kernel heap PTR	Kernel stack PTR	Kernel stack PTR (KASLR)
Severity	HIGH (7.8)	HIGH (7.8)	HIGH (7.8)	HIGH (7.8)
Kernel tested	5.10.15	4.19.19	6.7.1	6.7.1
Required privs	Unprivileged	Unprivileged	CAP_BPF	CAP_NET_ADMIN
Unpriv LPE	Not achievable	**Achieved**	Theoretical	N/A (needs capability)
Complexity	Medium	High	Very high	Low

Table 7.1: Comparison between all analyzed eBPF verifier exploits

Key differences: CVE-2021-3444 stands out as the only vulnerability where a **full unprivileged LPE** was achieved. While sharing the same root cause class as CVE-2021-3600 (32-bit truncation in `fixup_bpf_calls`), its exploitation on kernel 4.19.19, which lacks the `alu_limit` sanitization, demonstrates the critical importance of defense-in-depth: the same bug class is neutralized on 5.10+ but fully exploitable on 4.19. CVE-2024-26589 offers the simplest exploitation path (a 4-line patch fixes it) but requires `CAP_NET_ADMIN`. CVE-2023-52452 has the highest exploitation complexity, targeting the stack frame depth tracking subsystem.

7.5.2 Experimental Testing Matrix

CVE	Milestone	Expected Result	Observed Result
CVE-2021-3600	M1	Bounds mismatch visible	Confirmed
CVE-2021-3600	M2	OOB read primitive	Confirmed (15/15 reads)
CVE-2021-3600	M3	OOB write primitive	Confirmed (5/5 reliability)
CVE-2021-3600	M4	Unprivileged full LPE	Blocked by `alu_limit`
CVE-2021-3444	M1	Bounds mismatch visible	Confirmed
CVE-2021-3444	M2	OOB read (KASLR bypass)	Confirmed (`array_map_ops` leaked)
CVE-2021-3444	M3	OOB write (value_size corruption)	Confirmed (256→0x800)
CVE-2021-3444	M4	Full unprivileged LPE	**Achieved (uid=1001→uid=0)**
CVE-2023-52452	M1	Stack data disclosure	Confirmed (kernel pointers leaked)
CVE-2023-52452	M2	KASLR bypass via leak	Confirmed
CVE-2023-52452	M3	Type confusion from stack overwrite	Rejected (verifier keeps scalar)
CVE-2023-52452	M4	Controlled escalation feasibility	Partial: controlled corruption, no full LPE chain
CVE-2024-26589	M1	Verifier bypass on flow_keys	Confirmed (variable-offset accepted)
CVE-2024-26589	M2	OOB read beyond flow_keys	Confirmed (stack data leaked)
CVE-2024-26589	M3	Controlled OOB with map exfil	Confirmed (KASLR leak)
CVE-2024-26589	M4	OOB write / full analysis	Confirmed (stack corruption)

Table 7.2: Experimental matrix: expected vs observed outcomes for analyzed milestones

This matrix provides a compact view of the empirical results and complements the detailed per-CVE exploitation logs.

7.6 Limits and Failed Escalation Summary

Across all four CVEs, the strongest practical lesson is that proving a primitive and achieving stable unprivileged LPE are different milestones.

CVE-2021-3600 limits

- OOB read/write primitives are valid in privileged contexts.
- Unprivileged exploitation is blocked by layered mitigations, especially `alu_limit` pointer sanitization.
- Therefore, this CVE alone does not yield a robust unprivileged root escalation in the tested setup.

CVE-2021-3444 limits

- Full unprivileged LPE was achieved on kernel 4.19.19, but requires SMEP/SMAP/KPTI to be disabled for the ret2usr technique.
- The SLUB heap layout is non-deterministic: identifying the correct file descriptor for the corrupted map required iterating over all candidates.
- On kernels with `alu_limit` (5.x+), the same vulnerability class is neutralized, demonstrating the effectiveness of defense-in-depth.

CVE-2023-52452 limits

- Stack leak and controlled corruption are reproducible and technically strong.
- Direct type-confusion dereference paths are rejected by verifier checks.
- Full escalation would require additional chaining steps (precise stack mapping, controlled fake object placement, and execution redirection).

CVE-2024-26589 limits

- Full OOB read/write primitives on kernel stack are confirmed and reproducible.
- The vulnerability is not reachable from unprivileged users (requires `CAP_NET_ADMIN`).
- Full LPE via ROP chain is theoretically achievable but requires precise stack layout knowledge that varies with compiler optimizations and kernel call paths.
- Primary real-world impact: container escape when `CAP_NET_ADMIN` is granted.

Takeaway

- Documenting failed escalation attempts is essential for scientific rigor.
- A CVE can be highly relevant even when full LPE is not reached, if the primitive is well-characterized and repeatable.
- Different BPF program types expose different attack surfaces; the threat model varies based on required capabilities.

7.7 Final Thoughts

The four vulnerabilities analyzed in this report represent distinct but equally critical classes of verifier bugs: post-verification instruction rewriting flaws (CVE-2021-3600 and CVE-2021-3444), dynamic stack depth miscalculations (CVE-2023-52452), and missing pointer type restrictions (CVE-2024-26589). Together, they demonstrate that verifier safety failures span multiple subsystems and BPF program types. The successful full LPE achieved through CVE-2021-3444 on kernel 4.19.19 underscores how the absence of defense-in-depth mechanisms (such as `alu_limit`) can elevate a verifier logic bug from a theoretical concern to a practical root compromise. The research highlights that while the eBPF subsystem is a powerful tool for kernel-level extensibility, its security heavily relies on the correctness of the verifier's complex logic. While mitigations like `unprivileged_bpf_disabled=1` drastically reduce the attack surface for regular users, the widespread adoption of eBPF in privileged containers and cloud-native environments means that verifier logic flaws will continue to be a high-value target for kernel exploitation.